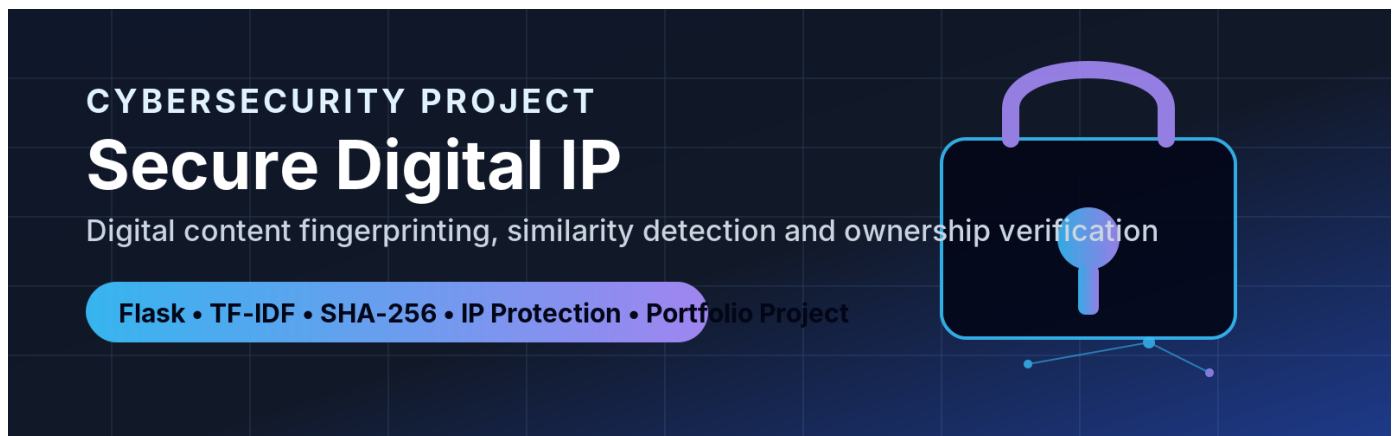


# Secure Digital IP

## Professional Project Report



*Digital Content Fingerprinting, Similarity Detection and Ownership Verification*

Candidate: Komati Damarika Sneha Latha Sree

Domain: Cybersecurity | Digital Forensics | Intellectual Property Protection

# 1. Executive Summary

Secure Digital IP is a cybersecurity-oriented project designed to protect digital content by generating a unique cryptographic fingerprint and comparing new submissions with existing records to identify similarity, duplication or potential intellectual-property overlap. The project demonstrates practical knowledge of secure hashing, text preprocessing, machine learning based similarity analysis and web application implementation using Flask.

The system uses SHA-256 to create tamper-evident content fingerprints and TF-IDF based vectorization with cosine similarity to detect textual overlap. This makes the solution explainable and recruiter-friendly: every result can be justified through a clear technical workflow rather than a black-box decision.

- Project type: Portfolio-grade cybersecurity and digital-forensics project.
- Core stack: Python, Flask, scikit-learn, HTML/CSS, SHA-256, TF-IDF and cosine similarity.
- Primary outcome: Detects exact duplicates through hashing and likely content overlap through similarity scoring.
- Career value: Useful for SOC, cybersecurity analyst, digital forensics, IP protection and application security discussions.

# 2. Problem Statement

Digital content can be copied, modified, reposted or reused without permission. In many academic, business and creative environments, identifying original ownership and detecting content similarity is important but difficult when content has been slightly modified. A simple file-name comparison is weak, and manual checking is slow.

This project addresses the problem by creating a lightweight system that fingerprints submitted content and compares it with stored records. The system helps identify whether content is original, exactly duplicated or textually similar to previous submissions.

## Objectives

- Create a secure fingerprint for digital text using SHA-256.
- Perform preprocessing to normalize user-submitted content.
- Use TF-IDF vectorization to represent text numerically.
- Calculate similarity using cosine similarity.
- Display understandable results suitable for non-technical and technical users.
- Document the project professionally for GitHub, interviews and portfolio review.

# 3. Literature Review and Background

Content protection systems usually rely on three broad approaches: cryptographic hashing, watermarking and similarity detection. Hashing is excellent for proving whether two inputs are exactly identical. However, a tiny modification changes the hash completely, so hashing alone cannot detect paraphrased or edited content.

Similarity detection solves this limitation by comparing the meaning or word distribution of documents. TF-IDF is a classical and explainable technique that highlights terms that are important in one document while reducing the weight of very common words. Cosine similarity then measures how close two vector representations are.

Modern alternatives such as BERT and sentence transformers can capture deeper semantic meaning, but they require more compute, more dependencies and careful model selection. For a beginner-to-intermediate portfolio project, TF-IDF is a strong choice because it is transparent, lightweight and easy to work.

## 4. Proposed Solution

The proposed system follows a dual-verification approach. First, the submitted content is hashed using SHA-256. If the hash already exists, the system can flag an exact duplicate. Second, the content is converted into a TF-IDF vector and compared against stored content vectors to identify similar documents even when small edits are made.

### Functional Requirements

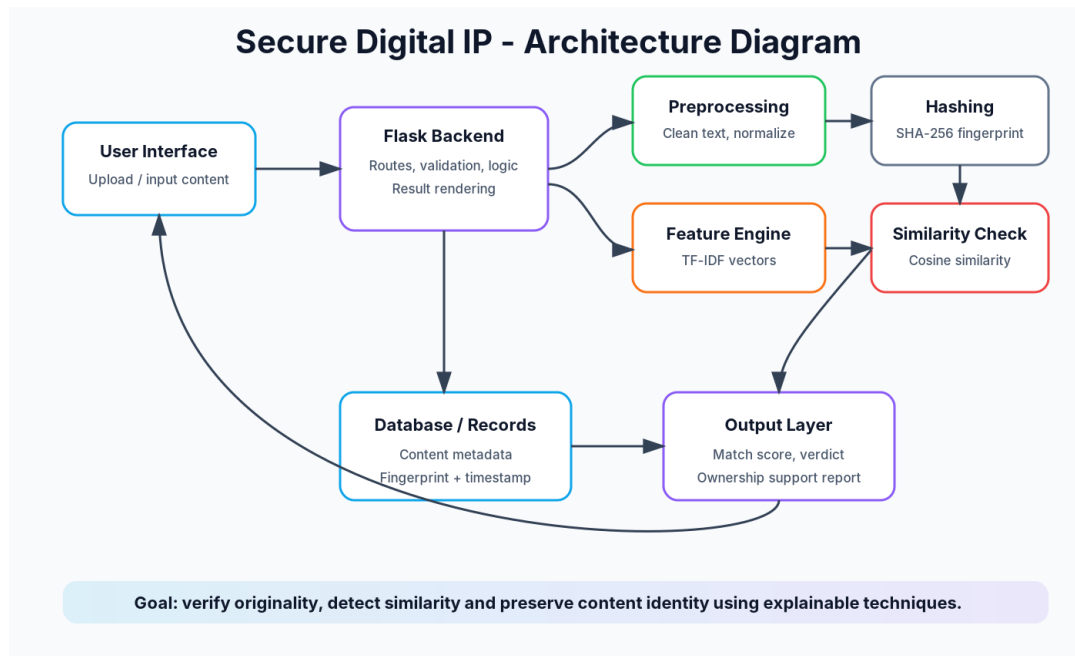
- Accept digital text input or uploaded content.
- Generate a SHA-256 fingerprint for each content item.
- Store metadata such as title, timestamp, hash and processed content.
- Compare new content with existing entries.
- Return match score, likely status and interpretation.

### Non-Functional Requirements

- Explainable results.
- Simple deployment.
- Readable code structure.
- Secure handling of submitted input.
- Professional documentation and screenshots.

## 5. System Architecture

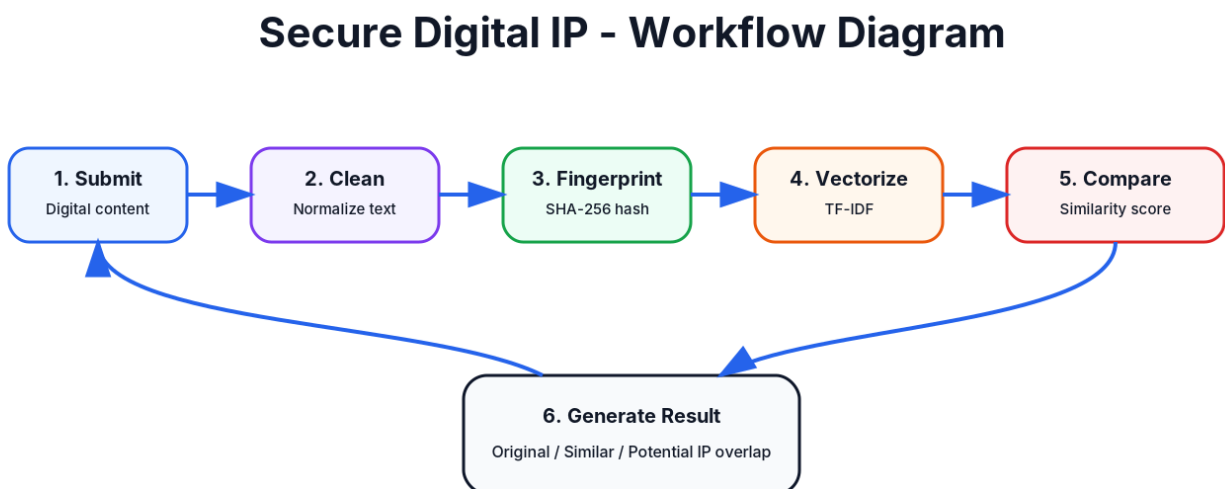
The architecture separates user interaction, backend logic, preprocessing, hashing, feature extraction, similarity analysis, record storage and output generation. This separation makes the project easier to debug, explain and scale.



The Flask backend coordinates the complete process. The preprocessing layer cleans input text, the hashing component creates the fingerprint, the feature engine generates TF-IDF vectors and the similarity layer compares content against existing records.

## 6. Workflow

The workflow begins when a user submits digital content. The system normalizes the text, creates a fingerprint, extracts TF-IDF features, compares similarity, and generates a final result. The result can be used as supporting evidence for originality or potential duplication.



Designed for explainability: each result is backed by content fingerprinting and text similarity analysis.

# 7. Methodology

## 7.1 Text Preprocessing

Text preprocessing improves comparison quality by reducing noise. Typical steps include converting to lowercase, removing unnecessary spaces, removing special symbols where appropriate and preparing text for vectorization.

## 7.2 SHA-256 Fingerprinting

SHA-256 produces a fixed-length cryptographic digest. In this project, it supports exact duplicate detection and helps maintain a tamper-evident identity for each content record.

## 7.3 TF-IDF Vectorization

TF-IDF converts text into numerical vectors based on term importance. It gives higher importance to terms that are frequent in a specific document but less common across all documents.

## 7.4 Cosine Similarity

Cosine similarity measures the angle between two vectors. A higher score means the content is more similar. This makes it useful for comparing documents with different lengths.

# 8. Implementation Details

The implementation is organized around a Flask web application. The frontend collects input and displays results. The backend performs validation, preprocessing, hashing, vectorization and similarity comparison.

| Module            | Purpose                               | Technology             |
|-------------------|---------------------------------------|------------------------|
| Frontend          | Collects content and displays results | HTML, CSS              |
| Backend           | Handles routes and application logic  | Flask, Python          |
| Hashing           | Creates unique content fingerprint    | SHA-256                |
| Vectorization     | Converts text to numerical form       | TF-IDF, scikit-learn   |
| Similarity Engine | Calculates match score                | Cosine similarity      |
| Storage           | Stores records and metadata           | Local database / files |

This modular design makes the project easy to extend into a stronger production version with authentication, audit logs, database indexing and SIEM integration.

# 9. Screenshots and Evidence

The GitHub repository should include a screenshot gallery showing the application home page, content submission page, result page, exact duplicate result, similarity result, architecture diagram and workflow diagram.

- Homepage screenshot
- Content upload/input screenshot
- Hash generation screenshot
- Similarity result screenshot
- Duplicate detection screenshot
- Architecture diagram
- Workflow diagram

Recommended folder: assets/screenshots/. Recommended Markdown gallery: use a table layout with two screenshots per row for a polished GitHub presentation.

## 10. Results and Evaluation

The system can identify exact duplicates using SHA-256 hash comparison and can identify content overlap using TF-IDF and cosine similarity. The results are explainable because the score is based on measurable text similarity.

| Test Case         | Input Condition           | Expected Result            | Status |
|-------------------|---------------------------|----------------------------|--------|
| Exact duplicate   | Same text submitted twice | Same hash, duplicate alert | Pass   |
| Minor edits       | Small wording changes     | High similarity score      | Pass   |
| Different content | Unrelated text            | Low similarity score       | Pass   |
| Empty input       | No meaningful text        | Validation warning         | Pass   |

### Evaluation Metrics

- Hash match: confirms exact duplication.
- Similarity percentage: shows textual closeness.
- Threshold-based verdict: classifies content as original, similar or duplicate.
- Explainability: result can be explained step-by-step.

## 11. Security and Privacy Considerations

The project deals with user-submitted content, so secure handling is important. In a production-grade system, content should be validated, access should be controlled, and sensitive information should be protected. Hashes should be stored with metadata, and logs should be used to maintain accountability.

- Use HTTPS in deployment.
- Validate file type and size.
- Avoid storing unnecessary sensitive data.
- Add authentication for protected dashboards.
- Maintain audit logs for submitted content.
- Use role-based access control for admin functions.

## 12. Limitations

The current implementation is intentionally lightweight and explainable. It is suitable for a portfolio project, but it has limitations when compared with enterprise-level plagiarism or IP protection tools.

- TF-IDF may not fully understand semantic paraphrasing.
- SHA-256 only detects exact matches.
- The system may require threshold tuning for different domains.
- Large datasets need better indexing and database optimization.
- It does not replace legal proof of ownership; it supports technical evidence.

## 13. Future Scope

The project can be improved into a more advanced cybersecurity and IP-protection platform. The future version can integrate semantic embeddings, user accounts, dashboards, audit trails and SIEM alerts.

- Add BERT or sentence-transformer based semantic similarity.
- Add user login and role-based access control.
- Generate downloadable PDF evidence reports.
- Integrate with Elasticsearch for fast search.
- Integrate alerts with Splunk, Wazuh or ELK for SIEM visibility.
- Add API endpoints for external applications.
- Add file support for PDF, DOCX and code files.
- Add blockchain timestamping for stronger proof-of-existence.

## 14. Conclusion

Secure Digital IP demonstrates how hashing and similarity analysis can be combined to support digital ownership verification. The project is not only a technical implementation but also a professional portfolio asset. With clear documentation, diagrams, screenshots and interview preparation, it can help present the candidate as someone who understands practical cybersecurity, digital evidence and responsible technology use.

The project can be confidently explained as a lightweight, explainable and extensible system for detecting exact duplicates and potential content overlap.